NeonCore 5G AI Roadmap

From a 5G lab to AI that finds its faults. Status as of 2026-11-12.

Goal: build a 5G core, generate labeled faults, and build AI that finds them, with numbers to prove it. Phases 0 to 6 built the lab. Stages W0 onward move it to Proxmox and add the AI, one stage at a time; each stage has a plan, an owner approval, an independent evaluation and a closing report.

Phases 0 to 6: the 5G lab (done)

Built first on a single-node K3s cluster in WSL2; the lab now runs on Proxmox (Stage L0).

PHASE	NAME	WHAT IT BUILT	STATUS
0	**Bare-metal validation** `phase0/`	Open5GS core and UERANSIM gNB/UE in docker-compose with pinned images, to prove the core, RAN and one subscriber work before Kubernetes.	Done
1	**K3s and Multus** `phase1/`	Single-node K3s with Multus CNI and local-path storage, the base for every later phase.	Done
2	**5G core and RAN on Kubernetes** `phase2/`	Open5GS network functions and UERANSIM as Deployments with headless Services in the `neoncore` namespace; a UE registers and pings through the UPF.	Done
3	**Monitoring and observability** `phase3/`	Prometheus, Grafana and kube-state-metrics in the `monitoring` namespace; AMF, SMF, UPF and PCF metrics, with a custom UPF build for the GTP-U packet counter.	Done
4	**Network tracing** `phase4/`	A tracer pod capturing N2, N3, N4 and SBI traffic at node level, with rotated pcaps on a persistent volume.	Done
5	**Terminal control center** `phase5/`	A Textual TUI to deploy, watch and tear down the lab, view pods and logs, and start captures.	Done
6	**Documentation and scenario automation** `phase6/`	README and DEVLOG rewrite, then automated signaling scenarios (registration, registration reject, deregistration) with one pcap per run. The DEVLOG calls the scenario work "Phase 7"; its code is in `phase6/`.	Done

Stages W0 to 12: Proxmox and AI

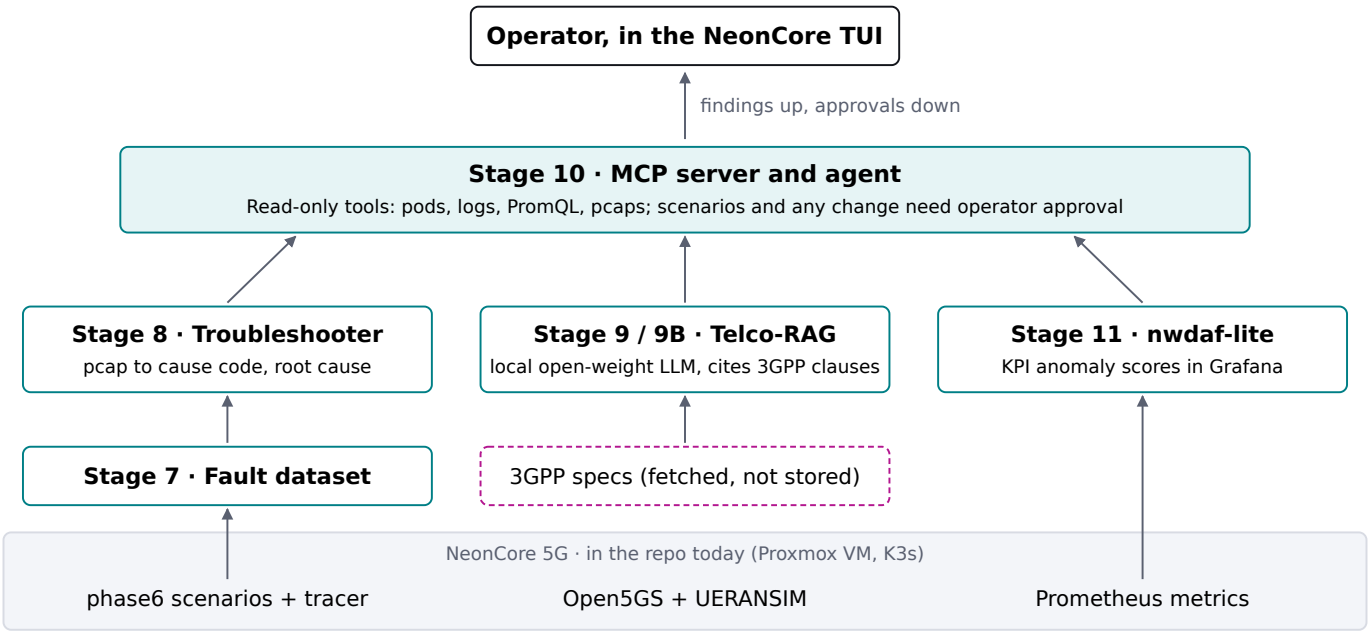
The order is fixed: W0, L0, 7, 8, 9, 10, 9B, 11, 12. A stage does not start until the one before it has closed.

STAGE	NAME	SHORT DESCRIPTION	STATUS
W0	**Agent workflow**	A lead session plus specialist agents (planner, developer, lab-ops, evaluator, telco and security reviewers, writers), and an approval gate that blocks building without an owner-approved plan. **Done when:** the gate tests pass.	Closed
L0	**Proxmox migration and repo polish** `infra/`	Move the lab to a Proxmox VM with Terraform and Ansible, one command to rebuild it from zero, LAN exposure closed, golden snapshot; README with a demo GIF and this roadmap. **Done when:** all pods are healthy, a UE pings, the three scenarios pass with valid pcaps, Grafana shows data, and the repo explains itself in 30 seconds.	Closed old-host retirement still open

7	**Fault injection and dataset** `phase7/`	Six labeled faults: wrong K/OPc, unknown DNN, wrong S-NSSAI, PCF down, SMF scaled to 0, UPF down. A generator loops scenario × fault × subscriber and writes pcaps, labels and a Prometheus snapshot per run. **Done when:** each fault is reproducible from the TUI with its own pcap, and at least 200 validated labeled runs exist.	**Next** after the approval-signing fix
8	**Call-flow extractor and troubleshooter** `phase8/`	A tshark extractor turns each pcap into a JSON call flow (NGAP and NAS messages, 5GMM/5GSM causes, timings). An LLM reads it and returns the failing step, cause code and root cause, scored against the labels. **Done when:** the JSON matches Wireshark for every fault, and accuracy per fault type is published.	**Planned**
9	**Telco-RAG, self-built and local** `phase9/`	A self-built RAG over the 3GPP specifications (TS 23.501, 23.502, 24.501, 38.413 to start), chunked by clause, with hybrid search, reranking and clause citations. Embeddings and answers come from an open-weight LLM running locally on a Mac mini (Apple M4 Pro) on the LAN, not a cloud API. Specs are fetched from 3GPP, never committed. **Done when:** questions return the right clause, scores on a GSMA Open-Telco subset plus 50 project questions are published, and the model server is reachable only from the lab.	**Planned**
10	**MCP server and agent** `phase10/`	An MCP server with read-only tools (pods, logs, PromQL, pcap summary) and an agent that combines the troubleshooter, Telco-RAG and MCP. Any change, including running a scenario, needs operator approval. An eval harness records task success, tool calls, cost and latency. **Done when:** the agent diagnoses a fault injected live, one command prints the eval table, and a 5-minute demo is linked.	**Planned**
9B	**Local vs API RAG comparison** `phase9/`	Run the same Telco-RAG pipeline with the Claude API as the answer model and compare it with the local model on the Stage 9 eval set: answer quality, clause accuracy, latency and cost per 1,000 queries. Runs after Stage 10 closes. **Done when:** a decision record compares local and API answers side by side.	**Planned**
11	**KPI anomaly detection (`nwdaf-lite`)** `phase11/`	A KPI dataset with labeled fault windows, an anomaly model served as a Kubernetes Deployment with an analytics API and a Grafana panel, MLflow tracking, a CI eval gate, and a mapping to NWDAF analytics types (TS 23.288). **Done when:** Grafana flags an injected fault, detection and false-alarm rates are published, and a worse model fails CI.	**Planned**
12	**Model study, red-team and reference architecture** `phase12/`	An open model on a rented GPU versus the API on the eval set (quality, latency, cost); red-teaming so a malicious log or SBI string cannot trigger a tool call; a GenAI-assisted NOC reference architecture and a 10-minute demo. **Done when:** the decision record and injection tests in CI exist, and the reference architecture is linked from the README.	**Planned**

Project specs built on NeonCore

Each AI stage is a new folder in the same repo, so the work reads as one system that grows, not separate demos. Every stage ships with an evaluation number.



NeonCore AI architecture, Stages 7 to 11. The agent sits on top and calls the three AI components as tools; it also reaches the live core through the MCP server, but never changes it without operator approval.

Design rules for all stages

- The agent reads freely but asks before any change, the same way the CLI already confirms deploy and teardown.
- Treat logs, pcaps and SBI payloads as untrusted input to the LLM.
- Record cost and latency next to accuracy; operators decide on all three.

Project ground rules

- One write-up per finished stage: the problem, a screenshot, the number, what broke.
- Commit in small steps with clear messages.
- Never use real operator pcaps, configs or IMSIs, or proprietary documents. NeonCore and synthetic data only.
- Set a monthly spending cap on API and cloud GPU accounts.

Stage definitions: `project/WORKFLOW.md`. Status: `project/STATUS.md`. Build history: `DEVLOG.md`. Built with AI agents under the owner's direction.